

Established – 1961

Subject: OOPS

SEVA SADAN'S
R. K. TALREJA COLLEGE
OF
ARTS, SCIENCE & COMMERCE
ULHASNAGAR – 421 003



CERTIFICATE

This is to certify that Mr./Ms. BHOOMI MISHRA of F.Y. Information Technology (FYIT) Roll No. 2541021 has satisfactorily completed the Object Oriented Programming using C++ Mini Project entitled THE VOTING SYSTEM during the academic year 2025 – 2026, as a part of the practical requirement. The project work is found to be satisfactory and is approved for submission.

PROF. INCHARGE

KUMODH KUKREJA

HEAD OF DEPT

PROF. LAXMI JESHWANI

INDEX

SR. NO.	CHAPTERS	PAGE NO.
1.	INTRODUCTION	1
2.	TOPIC DESCRIPTION	2
3.	OOP CONCEPT USED	3
4.	IMPLEMENTATION (CODE)	6
5.	OUTPUT	12
6.	CONCLUSION	14

INTRODUCTION

This project implements a simple Voting System in C++ that demonstrates the effective use of Object-Oriented Programming (OOP) concepts. The system allows eligible voters to cast votes for candidates, view election results, and declare the winner based on the highest number of votes received.

The program is designed using core OOP principles such as Abstraction, Encapsulation, Inheritance, and Polymorphism. An abstract base class Person is used to define common attributes like name and age, which are inherited by derived classes Candidate and Voter. The Election class manages candidate registration, vote casting, result display, and winner declaration.

The voting system ensures that only voters aged 18 or above are eligible to vote, maintaining basic election rules. A menu-driven interface is provided for user interaction, making the system easy to use and understand.

Overall, this project serves as a practical example of applying object-oriented design in C++ to build a real-world application such as an electronic voting system.

TOPIC DESCRIPTION

This project is a console-based Voting System developed using the C++ programming language. The main purpose of this system is to simulate a simple election process where eligible voters can cast their votes, results can be viewed, and the winner can be declared. The project is designed by applying important Object-Oriented Programming (OOP) concepts such as Abstraction, Encapsulation, Inheritance, and Polymorphism.

The system allows users to:

- View the list of candidates**
- Cast votes (only if the voter is eligible)**
- Display election results**
- Declare the winner based on maximum votes**

WORKING OF THE SYSTEM

- 1. The program starts by creating an Election object and adding predefined candidates.**
- 2. A menu-driven interface is displayed using a do-while loop and switch-case.**
- 3. When the user selects Cast Vote, voter details (name and age) are taken.**
- 4. The voter's eligibility is checked ($\text{age} \geq 18$).**
- 5. If eligible, the voter selects a candidate and the vote is recorded.**
- 6. The system can display:**
 - Total votes received by each candidate**
 - Winner of the election**
- 7. The program continues until the user chooses to exit.**

OOP CONCEPT USED

1. Abstraction

Abstraction is achieved using the abstract class Person. It hides implementation details and shows only essential features.

Person contains common attributes like name and age

It has a pure virtual function display(), making the class abstract

```
class Person {  
protected:  
    string name;  
    int age;  
public:  
    Person(string n, int a) : name(n), age(a) {}  
    virtual void display() = 0;  
};
```

2. Encapsulation

Encapsulation is implemented by keeping data members private and providing controlled access through public methods.

votes in Candidate class is private

Accessed only through addVote() and getVotes()

```
class Candidate : public Person {  
private:  
    int votes;
```

```
public:  
    void addVote();  
    int getVotes() const;  
};
```

3. Inheritance

Inheritance allows one class to acquire properties of another.

Candidate and Voter inherit from Person

This shows hierarchical inheritance

```
class Candidate : public Person { };  
class Voter : public Person { };
```

4. Polymorphism

Polymorphism is achieved through function overriding.

The display() function is overridden in both Candidate and Voter

Function call depends on the object type at runtime

```
void display() override {  
    cout << name << endl;  
}
```

5. Data Hiding

Sensitive data (like vote count) is protected from direct access.

Users cannot modify votes directly

This ensures data security and integrity

6. Dynamic Binding

Since display() is a virtual function, the correct version is called at runtime depending on the object.

7. Modular Design

The program is divided into multiple classes:

Person

Candidate

Voter

Election

This improves readability, reusability, and maintainability.

IMPLEMENTATION (CODE)

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;

// ===== ABSTRACTION
=====
class Person {
protected:
    string name;
    int age;

public:
    Person(string n, int a) : name(n), age(a) {}
    virtual void display() = 0;
};

// ===== HIERARCHICAL
INHERITANCE =====
class Candidate : public Person {
private:
    int votes; // ENCAPSULATION

public:
    Candidate(string n) : Person(n, 0), votes(0) {}

    void addVote() {
        votes++;
    }

    int getVotes() const {
        return votes;
    }
}
```



```

    string getName() const {
        return name;
    }

    void display() override { // POLYMORPHISM
        cout << name << endl;
    }
};

class Voter : public Person {
public:
    Voter(string n, int a) : Person(n, a) {}

    bool isEligible() const {
        return age >= 18;
    }

    void display() override {
        cout << "Voter Name: " << name << ", Age: " << age
        << endl;
    }
};

// ===== ELECTION CLASS
=====
class Election {
private:
    vector<Candidate> candidates;

public:
    void addCandidate(string name) {
        candidates.push_back(Candidate(name));
    }

    void showCandidates() {

```

```

    cout << "\n--- Candidates List ---\n";
    for (size_t i = 0; i < candidates.size(); i++) {
        cout << i + 1 << ". ";
        candidates[i].display();
    }
}

void castVote(int choice) {
    if (choice > 0 && choice <= candidates.size()) {
        candidates[choice - 1].addVote();
        cout << "Vote cast successfully!\n";
    } else {
        cout << "Invalid candidate choice!\n";
    }
}

// ===== SHOW RESULTS ONLY =====
void showResult() {
    cout << "\n--- Election Results ---\n";
    for (auto &c : candidates) {
        cout << c.getName() << " : " << c.getVotes() << "
votes\n";
    }
}

// ===== WINNER DECLARATION (SEPARATE)
=====
void declareWinner() {
    int maxVotes = 0;

    for (auto &c : candidates) {
        if (c.getVotes() > maxVotes) {
            maxVotes = c.getVotes();
        }
    }
}

```

```

    if (maxVotes == 0) {
        cout << "\nNo votes cast yet.\n";
        return;
    }

    cout << "\n--- Winner Declaration ---\n";
    bool first = true;

    for (auto &c : candidates) {
        if (c.getVotes() == maxVotes) {
            if (!first) cout << ", ";
            cout << c.getName();
            first = false;
        }
    }

    cout << " (with " << maxVotes << " votes)\n";
}

};

// ===== MAIN FUNCTION
=====

int main() {
    Election election;
    int choice;

    election.addCandidate("NARENDRA MODI");
    election.addCandidate("RAHUL GANDHI");
    election.addCandidate("MAMATA BANERJEE ");
    election.addCandidate("NITISH KUMAR");

    do {
        cout << "\n===== VOTING SYSTEM MENU
===== \n";
        cout << "1. Cast Vote\n";
        cout << "2. Show Results\n";
    } while (choice != 0);
}

```

```
cout << "3. Show Winner\n";
cout << "4. Exit\n";
cout << "Enter your choice: ";
cin >> choice;

switch (choice) {

case 1: {
    string voterName;
    int voterAge;

    cout << "Enter voter name: ";
    cin >> voterName;
    cout << "Enter voter age: ";
    cin >> voterAge;

    Voter voter(voterName, voterAge);

    if (!voter.isEligible()) {
        cout << "Not eligible to vote (Age must be 18 or
above)\n";
        break;
    }

    election.showCandidates();

    int voteChoice;
    cout << "Enter candidate number: ";
    cin >> voteChoice;

    election.castVote(voteChoice);
    break;
}

case 2:
    election.showResult();
```

```
        break;

    case 3:
        election.declareWinner();
        break;

    case 4:
        cout << "Exiting Voting System...\n";
        break;

    default:
        cout << "Invalid choice! Try again.\n";
    }

} while (choice != 4);

return 0;
}
```

OUTPUT

===== VOTING SYSTEM MENU =====

- 1. Cast Vote**
- 2. Show Results**
- 3. Show Winner**
- 4. Exit**

Enter your choice: 1

Enter voter name: AKASH

Enter voter age: 19

--- Candidates List ---

- 1. NARENDRA MODI**
- 2. RAHUL GANDHI**
- 3. MAMATA BANERJEE**
- 4. NITISH KUMAR**

Enter candidate number: 1

Vote cast successfully!

===== VOTING SYSTEM MENU =====

- 1. Cast Vote**
- 2. Show Results**
- 3. Show Winner**
- 4. Exit**

Enter your choice: 1

Enter voter name: ROHAN

Enter voter age: 22

--- Candidates List ---

- 1. NARENDRA MODI**
- 2. RAHUL GANDHI**
- 3. MAMATA BANERJEE**
- 4. NITISH KUMAR**

Enter candidate number: 2

Vote cast successfully!

===== VOTING SYSTEM MENU =====

- 1. Cast Vote**
- 2. Show Results**
- 3. Show Winner**
- 4. Exit**

Enter your choice: 2

--- Election Results ---

NARENDRA MODI : 1 votes

RAHUL GANDHI : 1 votes

MAMATA BANERJEE : 0 votes

NITISH KUMAR : 0 votes

===== VOTING SYSTEM MENU =====

- 1. Cast Vote**
- 2. Show Results**
- 3. Show Winner**
- 4. Exit**

Enter your choice: 3

--- Winner Declaration ---

NARENDRA MODI, RAHUL GANDHI (with 1 votes)

===== VOTING SYSTEM MENU =====

- 1. Cast Vote**
- 2. Show Results**
- 3. Show Winner**
- 4. Exit**

Enter your choice: 4

Exiting Voting System...

Enter voter name: ANKIT

Enter voter age: 16

Not eligible to vote (Age must be 18 or above)

CONCLUSION

The Voting System implemented in this project successfully demonstrates the practical application of Object-Oriented Programming (OOP) concepts in C++. By using abstraction, encapsulation, inheritance, and polymorphism, the program achieves a clean, modular, and well-organized design that closely models a real-world election process.

The use of an abstract base class (Person) helps in defining common properties for both voters and candidates while enforcing a consistent interface through virtual functions. Encapsulation ensures that sensitive data, such as the number of votes received by each candidate, is protected from unauthorized access, thereby maintaining data integrity. Inheritance enables code reusability and reduces redundancy, while polymorphism allows flexible and dynamic behavior through overridden functions.

The system also incorporates basic validation, such as checking voter eligibility based on age, which enhances reliability and correctness. The menu-driven approach improves user interaction and makes the program easy to operate. Separate functions for vote casting, result display, and winner declaration further enhance clarity and maintainability.

Overall, this project provides a strong foundation for understanding OOP principles and their real-world applications. With further enhancements such as data persistence, secure authentication, and graphical interfaces, the system can be extended into a more robust and scalable electronic voting solution.